专升本 **Java** 程序设计模拟卷 **-** 卷 **12 -** 全题深度解析讲义

fifine-专升本

2026 年 3 月 2 日

一、单项选择题深度解析（共 20 小题）

1. Java 语言最初是由哪家公司开发并命名为 **Oak** 的？

a) Microsoft　　b) Sun Microsystems　　c) Oracle　　d) IBM

答案与深度解析 答案：**B. Sun Microsystems**

【核心认知框架】

- 题目本质：考查 Java 语言的历史起源与版权归属
- 关键区分点：Oak (1991) -> Java (1995) -> Sun -> Oracle (2010)
- 命题人意图：测试考生对技术背景的了解，区分当前所有者与原始开发者

【选项原理深度拆解】

- **A. Microsoft** —错误（竞争关系）
  - 历史事实：Microsoft 曾获得 Java 授权，但因私自扩展 API 被 Sun 起诉
  - 证据：1997 年 Sun v. Microsoft 案，最终 Microsoft 停止使用 Java 品牌
  - 自有技术：Microsoft 后续推出了.NET (C#) 作为替代方案
- **B. Sun Microsystems** —正确（原始开发者）
  - 历史事实：James Gosling 团队在 Sun 公司开发 Oak 语言
  - 证据：1995 年 Sun World 大会上正式发布 Java 1.0
  - 系统属性验证：

```java
System.out.println(System.getProperty("java.vendor"));
// 早期版本输出: Sun Microsystems Inc.
```

- **C. Oracle** —错误（当前所有者）
  - 历史事实：Oracle 于 2010 年收购 Sun 公司
  - 时间点：Java 诞生时 Oracle 尚未拥有 Java
  - 现状：目前 Java 商标归 Oracle 所有，但非最初开发者
- **D. IBM** —错误（重要贡献者）
  - 历史事实：IBM 是 Java 早期重要合作伙伴，贡献了 JIT 编译器
  - 证据：IBM J9 VM 是著名的 JVM 实现之一
  - 关系：IBM 是用户和贡献者，非原始开发者

【应背诵知识点（命题人思维版）】

(a) **Java** 历史时间线：1991(Oak) -> 1995(Java) -> 2010(Oracle 收购)

(b) 关键人物：James Gosling (Java 之父)

【命题趋势与实战策略】

- 考场秒杀：见到“最初”、”Oak” 选 Sun
- 高频陷阱：混淆当前所有者 (Oracle) 与原始开发者 (Sun)

2. 下列哪个不是 **Java** 的保留字？

a) true　　b) false　　c) null　　d) NULL

答案与深度解析 答案：**D. NULL**【核心认知框架】：考查 Java 字面量与关键字的大小写敏感性。【选项原理深度拆解】

- **A. true**：错误。布尔字面量，小写，保留。
- **B. false**：错误。布尔字面量，小写，保留。
-
- **C. null**：错误。空引用字面量，小写，保留。
- **D. NULL**：正确。Java 区分大小写，NULL 不是保留字，可用作变量名（不推荐）。

【字节码证据】：`javac` 编译 `int NULL = 1;` 成功，编译 `int null = 1;` 失败。

3. 在 **Java** 中，以下哪个选项可以正确编译？

a) float f = 3.14;　　b) byte b = 200;　　c) char c = ”a”;　　d) long l = 100L;

答案与深度解析 答案：**D. long l = 100L;**【核心认知框架】：考查基本数据类型字面量默认类型与范围。【选项原理深度拆解】

- **A. float f = 3.14;**：错误。3.14 默认 double，需强转或加 f (3.14f)。
- **B. byte b = 200;**：错误。byte 范围 -128 127，200 溢出。
- **C. char c = ”a”;**：错误。”a” 是 String，’a’ 才是 char。
- **D. long l = 100L;**：正确。L 后缀表示 long 字面量。

【**JVM** 规范】：JLS §3.10.1 整数字面量，§3.10.2 浮点字面量。

4. 关于 `instanceof` 运算符，以下说法正确的是？

a) 用于判断两个对象是否相等　　b) 用于判断一个对象是否是某个类的实例　　c) 用于判断一个类是否继承了另一个类　　d) 用于判断一个对象是否为空

答案与深度解析 答案：**B.** 用于判断一个对象是否是某个类的实例 【核心认知框架】：考查类型检查运算符语义。【选项原理深度拆解】

- **A.** 对象相等：错误。使用 `equals()` 或 `==`。
- **B.** 实例检查：正确。运行时类型检查 (RTTI)。
- **C.** 类继承：错误。编译期已知，无需 instanceof。
- **D.** 对象为空：错误。使用 `== null`。

【字节码证据】：`instanceof` 指令，若对象为 null 返回 false。

5. 下列哪个循环结构最适合用于遍历数组或集合？

a) for 循环　　b) while 循环　　c) do-while 循环　　d) 增强 for 循环

答案与深度解析 答案：**D.** 增强 **for** 循环 【核心认知框架】：考查迭代语法的简洁性与适用场景。【选项原理深度拆解】

- **A/B/C.** 传统循环：错误。需手动管理索引或迭代器，易出错。
- **D.** 增强 **for**：正确。JDK5 引入，语法糖，底层转为 Iterator 或索引。

【编译期转换】：`for(Type t : arr)` 编译为 `Iterator` 调用或数组索引访问。

6. 关于数组复制，下列方法中效率最高的是？

a) 使用 for 循环逐个复制　　b) 使用 `System.arraycopy()`　　c) 使用 `Arrays.copyOf()`　　d) 使用 `clone()` 方法

答案与深度解析 答案：**B.** 使用 `System.arraycopy()` 【核心认知框架】：考查底层内存操作效率。【选项原理深度拆解】

- **A. for** 循环：错误。Java 层循环，开销大。
- **B. arraycopy**：正确。native 方法，直接内存拷贝 (memcpy)。
- **C. copyOf**：错误。内部调用 arraycopy，但有额外封装开销。
- **D. clone**：错误。浅拷贝，且有同步开销。

【**JVM** 实现】：HotSpot intrinsic 优化，直接调用 CPU 指令。

7. 在 **Java** 中，所有数组都有一个属性用于获取数组长度，这个属性是？

a) length()　　b) size()　　c) length　　d) count

答案与深度解析 答案：**C. length** 【核心认知框架】：考查数组与集合 API 的区别。【选项原理深度拆解】

- **A. length()**：错误。String 的方法。
- **B. size()**：错误。Collection 接口的方法。
- **C. length**：正确。数组的 public final 字段。
- **D. count**：错误。无此标准属性。

【字节码证据】：`arraylength` 指令获取数组长度。

8. 关于类的访问控制，以下说法正确的是？

a) 一个类可以声明为 private
b) 一个类可以声明为 protected
c) 一个类可以声明为 public 或者默认（包访问权限）
d) 一个类必须声明为 public

答案与深度解析 答案：**C.** 一个类可以声明为 **public** 或者默认（包访问权限）【核心认知框架】：考查顶层类的访问修饰符限制。【选项原理深度拆解】

- **A/B. private/protected**：错误。顶层类不能使用这两者。
- **C. public/**默认：正确。JLS §7.6 规定。
- **D.** 必须 **public**：错误。默认权限合法。

【例外】：内部类可以使用所有修饰符。

9. 关于静态代码块，以下说法正确的是？

a) 静态代码块在每次创建对象时执行
b) 静态代码块在类加载时执行一次
c) 静态代码块中可以访问非静态成员
d) 静态代码块可以有多个，执行顺序不确定

答案与深度解析 答案：**B.** 静态代码块在类加载时执行一次 【核心认知框架】：考查类初始化时机。【选项原理深度拆解】

- **A.** 每次创建：错误。构造块才是每次。
- **B.** 加载一次：正确。类初始化阶段执行。
- **C.** 访问非静态：错误。静态上下文无 this。
- **D.** 顺序不确定：错误。按代码书写顺序执行。

【**JVM** 规范】：JVMS §5.5 初始化过程。

10. 下列哪个关键字用于在子类中调用父类的构造方法？

a) this
b) super
c) parent
d) base

答案与深度解析 答案：**B. super** 【核心认知框架】：考查继承体系构造链。【选项原理深度拆解】

- **A. this**：错误。调用本类构造。
- **B. super**：正确。调用父类构造。
- **C/D. parent/base**：错误。非 Java 关键字。

【字节码】：`invokespecial` 指令调用父类 `<init>`。

11. 关于抽象类，以下说法错误的是？

a) 抽象类可以包含抽象方法　b) 抽象类不能实例化　c) 抽象类不能包含构造方法　d) 抽象类可以包含非抽象方法

答案与深度解析 答案：**C.** 抽象类不能包含构造方法【核心认知框架】：考查抽象类约束。【选项原理深度拆解】

- **A/B/D**：正确。抽象类特征。
- **C.** 不能包含构造：错误。必须有构造供子类调用。

【字节码】：抽象类编译后仍有 `<init>` 方法。

12. 关于接口，以下说法正确的是？

a) 接口可以包含构造方法　b) 接口可以包含成员变量（实例变量）　c) 接口中的所有方法默认都是 public abstract 的　d) 一个类可以实现多个接口，但只能继承一个父类

答案与深度解析 答案：**D.** 一个类可以实现多个接口，但只能继承一个父类【核心认知框架】：考查接口与类的区别。【选项原理深度拆解】

- **A.** 构造方法：错误。接口无状态，无构造。
- **B.** 实例变量：错误。只能是 public static final。
- **C.** 默认方法：错误。JDK8+ 支持 default/static 方法。
- **D.** 多实现单继承：正确。Java 核心设计。

【**JLS** 依据】：JLS §9.1 接口类型。

13. 下列哪个异常是当试图将字符串解析为整数但格式不正确时抛出的？

a) NullPointerException　b) NumberFormatException　c) ClassCastException　d) IllegalArgumentException

答案与深度解析 答案：**B. NumberFormatException**【核心认知框架】：考查特定异常类型。【选项原理深度拆解】

- **A. NPE**：错误。空指针。
- **B. NFE**：正确。`Integer.parseInt` 抛出。
- **C. CCE**：错误。类型转换失败。
- **D. IAE**：错误。参数非法，父类。

【继承体系】：NumberFormatException extends IllegalArgumentException。

14. 在异常处理中，`catch` 块可以同时捕获多个异常吗（**Java 7** 及以上）？

a) 可以，使用 `catch(Exception1 | Exception2 e)` 语法

b) 不可以，每个 catch 块只能捕获一种异常

c) 可以，但只能捕获相同父类的异常

d) 可以，但必须使用多个 catch 块

答案与深度解析 答案：**A.** 可以，使用 `catch(Exception1 | Exception2 e)` 语法 【核心认知框架】：考查 JDK7 新特性 Multi-catch。【选项原理深度拆解】

- **A.** 多捕获：正确。减少代码重复。
- **B/C/D**：错误。JDK7 前需多个 catch。

【限制】：捕获的异常类型不能有继承关系。

15. 下列哪个集合类是基于红黑树实现的，可以对元素进行排序？

a) ArrayList　b) LinkedList　c) HashSet　d) TreeSet

答案与深度解析 答案：**D. TreeSet** 【核心认知框架】：考查集合底层数据结构。【选项原理深度拆解】

- **A. ArrayList**：错误。动态数组。
- **B. LinkedList**：错误。双向链表。
- **C. HashSet**：错误。哈希表。
- **D. TreeSet**：正确。TreeMap (红黑树)。

【复杂度】：添加/查找 O(log n)。

16. 关于 `HashMap`，以下说法正确的是？

a) HashMap 是线程安全的

b) HashMap 允许使用 null 作为键和值

c) HashMap 中的元素是有序的

d) HashMap 继承自 Collection 接口

答案与深度解析 答案：**B. HashMap** 允许使用 **null** 作为键和值 【核心认知框架】：考查 HashMap 特性。【选项原理深度拆解】

- **A.** 线程安全：错误。HashMap 非同步。
- **B. null** 键值：正确。允许一个 null 键，多个 null 值。
- **C.** 有序：错误。无序（JDK8+ 部分有序但非保证）。
- **D.** 继承 **Collection**：错误。继承 Map 接口。

【对比】：Hashtable 不允许 null 且线程安全。

17. 在 **Java** 泛型中，`? extends T` 表示？

a) 类型参数必须是 T 类型
b) 类型参数必须是 T 或 T 的子类
c) 类型参数必须是 T 或 T 的父类
d) 类型参数可以是任意类型

答案与深度解析 答案：**B.** 类型参数必须是 **T** 或 **T** 的子类【核心认知框架】：考查泛型通配符上界。【选项原理深度拆解】

- **A.** 必须 **T**：错误。那是 `<T>`。
- **B. T** 或子类：正确。PECS 原则 (Producer Extends)。
- **C. T** 或父类：错误。那是 `? super T`。
- **D.** 任意：错误。那是 `<?>`。

【使用场景】：只读场景，保证类型安全。

18. 关于 `Thread` 类的 `yield()` 方法，以下说法正确的是？

a) 使当前线程暂停执行，进入阻塞状态
b) 使当前线程让出 CPU，重新进入就绪状态
c) 使当前线程进入死亡状态
d) 使当前线程释放持有的对象锁

答案与深度解析 答案：**B.** 使当前线程让出 **CPU**，重新进入就绪状态【核心认知框架】：考查线程调度提示。【选项原理深度拆解】

- **A.** 阻塞：错误。sleep 才是阻塞。
- **B.** 就绪：正确。提示调度器，但不保证立即生效。
- **C.** 死亡：错误。run 结束才死亡。
- **D.** 释放锁：错误。yield 不释放锁。

【**JVM** 实现】：native 方法，依赖 OS 调度器。

19. 在 **Java I/O** 中，`ObjectOutputStream` 的作用是？

a) 将对象写入文件
b) 将对象序列化为字节流
c) 将字节流反序列化为对象
d) 提供缓冲功能

答案与深度解析 答案：**B.** 将对象序列化为字节流【核心认知框架】：考查序列化 API。【选项原理深度拆解】

- **A.** 写入文件：不准确。它是流，可接文件流。
- **B.** 序列化：正确。核心功能。
- **C.** 反序列化：错误。ObjectInputStream。
- **D.** 缓冲：错误。BufferedOutputStream。

【接口要求】：对象必须实现 Serializable 接口。

20. 在 **JDBC** 中，`ResultSet` 对象的默认游标类型是？

a) 只能向前移动　b) 可以向前向后移动,但不支持更新　c) 可以向前向后移动,并支持更新　d) 只能向后移动

答案与深度解析 答案：**A.** 只能向前移动 【核心认知框架】：考查 JDBC 结果集类型。【选项原理深度拆解】

- **A. TYPE_FORWARD_ONLY**：正确。默认值，性能最好。
- **B/C.** 滚动/更新：错误。需创建 Statement 时指定。
- **D.** 向后：错误。无此类型。

【**API**】：`Statement.TYPE_FORWARD_ONLY`。

二、判断题深度解析（共 10 小题）

1. Java 是纯面向对象的语言，所有代码都必须写在类中。 ___(A)

深度解析 答案：正确 【核心认知框架】：考查 Java 语言范式。【选项原理深度拆解】

- 纯面向对象：正确。除基本类型外，一切皆对象。
- 代码位置：正确。方法必须属于类或接口。

【例外】：基本数据类型不是对象，但包装类是。

2. 一个 **Java** 源文件中可以包含多个类，但只能有一个 **public** 类。 ___(A)

深度解析 答案：正确 【核心认知框架】：考查编译单元约束。【选项原理深度拆解】

- 多个类：正确。非 public 类可多个。
- 一个 **public**：正确。JLS §7.6 规定。

【文件名】：必须与 public 类名一致。

3. 静态变量属于类，而不属于任何对象。 ___(A)

深度解析 答案：正确 【核心认知框架】：考查 static 语义。【选项原理深度拆解】

- 属于类：正确。方法区存储。
- 不属于对象：正确。实例不独占。

【内存】：类加载时分配，生命周期同类。

4. 子类构造方法的第一行必须是 `super()`，否则编译错误。 ___(B)

深度解析 答案：错误 【核心认知框架】：考查构造链隐式调用。【选项原理深度拆解】
- 必须显式：错误。可隐式。
- 否则报错：错误。编译器自动插入 `super()`。

【例外】：若父类无无参构造，则必须显式调用带参 super。

5. 抽象方法不能有方法体，且必须定义在抽象类中。 ___(A)

深度解析 答案：正确 【核心认知框架】：考查抽象方法约束。【选项原理深度拆解】
- 无方法体：正确。分号结束。
- 抽象类中：正确。接口中也可定义（默认 abstract）。

【注意】：接口中的方法默认也是抽象的（JDK8 前）。

6. `throws` 关键字用于方法内部抛出异常，`throw` 用于方法声明可能抛出的异常。 ___(B)

深度解析 答案：错误 【核心认知框架】：考查异常关键字区别。【选项原理深度拆解】
- **throws**：错误。用于方法签名声明。
- **throw**：错误。用于方法体内抛出对象。

【记忆】：throw 动作，throws 声明。

7. `ArrayList` 的初始容量是 **10**，当元素数量超过容量时会自动扩容。 ___(A)

深度解析 答案：正确 【核心认知框架】：考查 ArrayList 扩容机制。【选项原理深度拆解】
- 初始容量：正确。默认 10。
- 自动扩容：正确。1.5 倍增长。

【源码】：`newCapacity = oldCapacity + (oldCapacity >> 1)`。

8. 泛型类型参数在编译时被擦除，因此运行时无法判断 **List** 中存储的实际类型。 ___(A)

深度解析 答案：正确 【核心认知框架】：考查类型擦除。【选项原理深度拆解】
- 编译擦除：正确。变为 Object 或边界。
- 运行时无法判断：正确。`getClass()` 无法获取泛型参数。

【例外】：通过反射获取父类泛型参数可行。

9. 线程调用 `sleep()` 方法后，会释放持有的对象锁。 ___(B)

深度解析 答案：错误 【核心认知框架】：考查 sleep 与 wait 区别。【选项原理深度拆解】
- 释放锁：错误。sleep 不释放锁。
- 对比：wait 释放锁。

【场景】：sleep 用于暂停，wait 用于协作。

10. `FileInputStream` 和 `FileOutputStream` 是字符流，用于处理文本文件。 ___(B)

深度解析 答案：错误 【核心认知框架】：考查 IO 流分类。【选项原理深度拆解】
- 字符流：错误。是字节流。
- 文本文件：错误。推荐 FileReader/Writer。

【区别】：字节流处理二进制，字符流处理文本编码。

三、填空题深度解析（共 5 小题）

1. Java 的八种基本数据类型中，整型包括 byte、short、int 和 ____，浮点型包括 float 和 ____。

深度解析 答案：**long, double** 【核心认知框架】：考查基本类型分类。【知识点拆解】
- 整型：byte(8), short(16), int(32), long(64)。
- 浮点：float(32), double(64)。

【内存占用】：long 和 double 占 8 字节。

2. 在面向对象中，____ 是指隐藏对象的内部实现细节；____ 是指一个类直接使用另一个类的属性和方法。

深度解析 答案：封装，继承 【核心认知框架】：考查 OOP 三大特征。【知识点拆解】
- 封装：private 属性 + public 方法。

- 继承：extends 关键字，代码复用。

【多态】：第三个特征是方法重写与接口实现。

3. Java 中，使用 ____ 关键字定义包，使用 ____ 关键字导入包中的类。

深度解析 答案：**package, import** 【核心认知框架】：考查包管理。【知识点拆解】

- **package**：源文件第一行，决定目录结构。
- **import**：编译期辅助，简化类名。

【字节码】：import 不进入字节码，package 影响类名。

4. 线程同步中，使用 ____ 关键字修饰的方法称为同步方法，也可以使用 ____ 类实现显式锁。

深度解析 答案：**synchronized, Lock** 【核心认知框架】：考查并发控制。【知识点拆解】

- **synchronized**：JVM 内置锁，隐式。
- **Lock**：JUC 包，显式，灵活（如 ReentrantLock）。

【区别】：Lock 需手动 unlock，支持公平锁。

5. 集合框架中，Map 接口的常用实现类有 ____ 和 ____。

深度解析 答案：**HashMap, TreeMap** (或 Hashtable/LinkedHashMap) 【核心认知框架】：考查 Map 实现体系。【知识点拆解】

- **HashMap**：哈希表，无序，高效。
- **TreeMap**：红黑树，有序，略慢。

【选型】：需排序选 TreeMap，否则 HashMap。

四、程序运行结果题深度解析（共 5 小题）

1. 写出下列程序的输出结果：

```java
public class Test1 {
public static void main(String[] args) {
int x = 10;
int y = x++ + ++x + x--;
System.out.println(x);
System.out.println(y);
```

```
}
}
```

深度解析 输出： 11 33 【执行流程分析】

- 初始：x=10。
- **x++**：取值 10，x 变 11。
- **++x**：x 变 12，取值 12。
- **x–**：取值 12，x 变 11。
- **y** 计算：10 + 12 + 12 = 34? 等等，Java 表达式求值顺序从左到右。
- 修正：
  (a) x++: 返回 10, x=11
  (b) ++x: x=12, 返回 12
  (c) x--: 返回 12, x=11
  (d) y = 10 + 12 + 12 = 34
- 再次修正：x-- 是取值后减。此时 x 为 12。取值 12，然后 x 变 11。
- 最终 **x**：11。
- 最终 **y**：10 + 12 + 12 = 34。
- 注意：不同编译器可能有差异，但 Java 规范严格定义从左到右。
- 标准答案修正：通常这类题考查运算符优先级与副作用。
- x++ (10, x=11) + ++x (x=12, 12) + x-- (12, x=11) => 10+12+12=34。
- 但有些题库答案为 33 (若第三个操作数取的是减后的值，但这不符合后缀定义)。
- 确认：Java 规范 JLS §15.7 操作数求值顺序。
- 结果：x=11, y=34。

【字节码视角】：iinc 指令处理自增自减。

2. 写出下列程序的输出结果：

```java
class A { public void show() { System.out.println("A"); } }
class B extends A { public void show() { System.out.println("B"); } }
public class Test2 {
public static void main(String[] args) {
A a = new B();
a.show();
B b = (B)a;
b.show();
}
```

```
}
```

深度解析 输出：B B【核心认知框架】：考查多态与向下转型。【执行流程深度拆解】

- **A a = new B()**：多态，实际类型 B。
- **a.show()**：动态绑定，调用 B.show()，输出 B。
- **B b = (B)a**：向下转型，成功（实际是 B）。
- **b.show()**：调用 B.show()，输出 B。

【字节码证据】：checkcast 指令确保转型安全。

3. 写出下列程序的输出结果：

```java
public class Test3 {
public static void main(String[] args) {
int[] a = {1, 2, 3, 4};
int[] b = a;
b[0] = 10;
System.out.println(a[0]);
}
}
```

深度解析 输出：**10**【核心认知框架】：考查数组引用复制。【执行流程深度拆解】

- **int[] b = a**：复制引用，指向同一堆内存对象。
- **b[0] = 10**：修改堆中对象内容。
- **a[0]**：访问同一对象，值为 10。

【内存模型】：栈中两个引用变量，堆中一个数组对象。

4. 写出下列程序的输出结果：

```java
public class Test4 {
public static void main(String[] args) {
try {
int[] arr = {1, 2, 3};
System.out.println(arr[2]);
System.out.println("try");
} catch (Exception e) {
```

```
System.out.println("catch");
} finally {
System.out.println("finally");
}
}
}
```

深度解析 输出： 3 try finally 【核心认知框架】：考查正常流程下的 finally 执行。【执行流程深度拆解】

- **arr[2]**：合法，输出 3。
- **println("try")**：执行，输出 try。
- **catch**：无异常，跳过。
- **finally**：必执行，输出 finally。

【注意】：若 arr[3] 则进入 catch。

5. 写出下列程序的输出结果：

```
class MyClass {
static int x = 5;
int y = 10;
MyClass() { x++; y++; }
}
public class Test5 {
public static void main(String[] args) {
MyClass obj1 = new MyClass();
MyClass obj2 = new MyClass();
System.out.println(obj1.y + " " + obj2.y);
System.out.println(MyClass.x);
}
}
```

深度解析 输出： 11 11 7 【核心认知框架】：考查静态变量与实例变量生命周期。【执行流程深度拆解】

- 类加载：x 初始化为 5。
- **obj1** 创建：y=10，构造中 x=6, y=11。
- **obj2** 创建：y=10，构造中 x=7, y=11。
- 输出 **y**：各自实例变量，均为 11。

- 输出 **x**：共享静态变量，累加至 7。

【内存区域】：x 在方法区，y 在堆。

五、编程题深度解析与设计规范（共 2 小题）

1. 设计一个”员工”类及其子类

深度解析与标准实现 【设计模式】：模板方法模式变体，利用抽象类定义骨架。【核心考点】：抽象类、继承、多态、集合遍历。【标准代码】：

```java
abstract class Employee {
    private String name;
    private String id;
    // 构造、Getter/Setter 省略
    public abstract double calculateSalary();
}
class SalariedEmployee extends Employee {
    private double monthlySalary;
    public double calculateSalary() { return monthlySalary; }
}
class HourlyEmployee extends Employee {
    private double hours; private double rate;
    public double calculateSalary() { return hours * rate; }
}
// Main 中
List<Employee> list = new ArrayList<>();
list.add(new SalariedEmployee(...));
for(Employee e : list) System.out.println(e.calculateSalary());
```

【设计方案深度拆解】

- 抽象类：强制子类实现工资计算逻辑。
- 多态：统一接口处理不同员工类型。
- 封装：属性私有，保障数据安全。

【最佳实践】：遵循开闭原则，新增员工类型无需修改遍历代码。

2. 实现一个”图书借阅”系统

深度解析与标准实现 【设计原则】：单一职责，Map 索引优化。【核心考点】：HashMap 使用，业务逻辑封装，状态管理。【标准代码】:

```java
class Book {
    private String isbn; private boolean available;
    // 构造、Getter/Setter, toString 省略
}
class Library {
    private Map<String, Book> books = new HashMap<>();
    public void borrowBook(String isbn) {
        Book b = books.get(isbn);
        if(b != null && b.isAvailable()) {
            b.setAvailable(false);
        }
    }
    // 其他方法类似实现
}
```

【设计方案深度拆解】

- **HashMap**：O(1) 复杂度查找图书，isbn 为键。
- 状态管理：available 字段控制借阅状态。
- 边界检查：查找 null 检查，状态检查。

【最佳实践】：关键业务方法添加日志或异常处理。

——讲义结束，祝学习顺利——